

메모리 분석 우회 기법 연구 및 개선방안 도출

Team. MalGround

Index

1

개요

- 1) 팀 소개
- 2) Motivation

2

방법론

3

PoC

- 1) Demo
- 2) 우회 가능 방법

4

방안제시

- 1) procdump
- 2) psscan
- 3) 그 외
- 4) 교차 검증 도구

5

결과

- 1) 학술대회 발표
- 2) VAC
- 3) CFP
- 4) 선행 연구
- 5) 추가 연구
- 6) 역할
- 7) WBS

1. 개요

팀 소개



디지털 포렌식
김종민 멘토님



디지털 포렌식
김종현 멘토님



디지털 포렌식
PM 이한얼



디지털 포렌식
김태우



디지털 포렌식
전은진



보안 제품 개발
김한솔



보안 제품 개발
한지혁

1. 개요

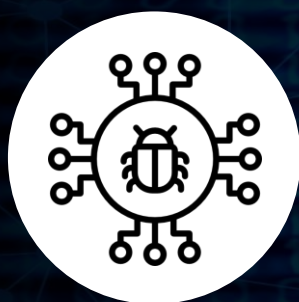
Motivation

7대 사이버 공격 전망 2019



다양한 경로를 통한
크립토재킹 확산

[Ahnlab]



소프트웨어 공급망 대상
사이버 공격 증가

[BITSCAN]



소프트웨어 공급망 대상
악성코드 유포

[EST security]



악성 행위 탐지를
우회하는 공격 기법 진화

[KISA]



사물인터넷을 겨냥한
신종 사이버 위협

[INCA]



고급 지속적 위협
APT 공격

[HAURI]



제로데이 보안취약점을
겨냥한 공격

[NSHC]

1. 개요

Motivation



1. 개요

Motivation

Olympic Destroyer

```
C:\#volatility-master>vol.py -f C:\Olympic.vmem --profile=Win7SP0x86 psscan
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name          PID  PPID  PDB      Time created      Time exited
-----
C:\#volatility-master>.
```

?



Ticket
Manager

1. 개요

Motivation

정상 파일

```
C:\#volatility-master>vol.py -f 0:\proc.vmem --profile=Win7SP0x64 psscan
```

Volatility Foundation Volatility Framework 2.6

Offset(P)	Name	PID	PPID	PDB	Time created	Time e
0x000000001d603b30	WmiPrvSE.exe	3180	656	0x0000000015e06000	2018-10-11 12:23:59 UTC+0000	
0x000000001d831280	PrintIsolation	3116	656	0x0000000015e06000	2018-10-11 12:23:46 UTC+0000	
0x000000001d837b30	node.exe	3168	2856	0x00000000123c2000	2018-10-11 12:23:47 UTC+0000	
0x000000001d863b30	SearchIndexer.	3288	496	0x000000001095f000	2018-10-11 12:23:47 UTC+0000	
0x000000001d8b0630	wmpnetwk.exe	3488	496	0x000000001aa15000	2018-10-11 12:23:48 UTC+0000	
0x000000001d8fab30	SearchFilterHo	3616	3288	0x000000000624a000	2018-10-11 12:23:49 UTC+0000	
0x000000001d8fcb30	svchost.exe	3648	496	0x000000001498c000	2018-10-11 12:23:49 UTC+0000	
0x000000001d9c68c0	GoogleUpdate.e	3944	2772	0x0000000011056000	2018-10-11 12:23:53 UTC+0000	
0x000000001d9cd5b0	GoogleUpdate.e	3976	2772	0x0000000000617000	2018-10-11 12:23:54 UTC+0000	
0x000000001d9d7b30	calc.exe	3216	2336	0x0000000017c87000	2018-10-11 12:23:58 UTC+0000	
0x000000001d9ebb30	notepad.exe	4080	2336	0x000000000fc38000	2018-10-11 12:23:57 UTC+0000	
0x000000001da11b30	explorer.exe	2336	2268	0x0000000015fec000	2018-10-11 12:23:40 UTC+0000	
0x000000001da1bb30	vmtoolsd.exe	2644	2336	0x000000001773d000	2018-10-11 12:23:41 UTC+0000	
0x000000001da45b30	CrossEXService	2460	1492	0x000000000cf6d000	2018-10-11 12:23:40 UTC+0000	
0x000000001dab9b30	taskeng.exe	2692	920	0x0000000009ae2000	2018-10-11 12:23:41 UTC+0000	
0x000000001dabb600	IMG5F50start_x	2752	1524	0x000000000081a000	2018-10-11 12:23:42 UTC+0000	
0x000000001dabbb30	IMG5F50start_x	2740	1524	0x0000000000ea1c000	2018-10-11 12:23:42 UTC+0000	
0x000000001daf8b30	AnySign4PC.exe	2840	2668	0x00000000005acb000	2018-10-11 12:23:42 UTC+0000	



Ticket Manager

1. 개요

Motivation

Olympic Destroyer

```
C:\#volatility-master>vol.py -f C:\Olympic.vmem --profile=Win7SP0x86 psscan
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name          PID  PPID  PDB      Time created      Time exited
-----
C:\#volatility-master>.
```

?



Ticket Manager

1. 개요

Motivation

정상 파일

```
C:\wvolatility-master>vol.py -f 0x0000000000000000 --profile=Win7SP0x64 psscan
```

Volatility Foundation Volatility Framework 2.6

Offset(P)	Name	PID	PPID	PDB	Time created	Time e
0x000000001d603b30	WmiPrivSE.exe	3180	656	0x0000000015e06000	2018-10-11 12:23:59 UTC+0000	
0x000000001d831280	PrintIsolation	3116	656	0x0000000015e06000	2018-10-11 12:23:46 UTC+0000	
0x000000001d837b30	node.exe	3168	2856	0x00000000123c2000	2018-10-11 12:23:47 UTC+0000	
0x000000001d863b30	SearchIndexer	3288	456	0x000000001095f000	2018-10-11 12:23:47 UTC+0000	
0x000000001d8b0630	WmiNet.exe	3488	706	0x000000001a150000	2018-10-11 12:23:48 UTC+0000	
0x000000001d8fab30	SearchIndexer	3616	3268	0x0000000010b54a000	2018-10-11 12:23:49 UTC+0000	
0x000000001d8fcb30	chost.exe	3648	496	0x0000000014182000	2018-10-11 12:23:49 UTC+0000	
0x000000001d9c68c0	GoogleUpdate.e	3944	2772	0x0000000011056000	2018-10-11 12:23:53 UTC+0000	
0x000000001d9cd5b0	GoogleUpdate.e	3976	2772	0x0000000000617000	2018-10-11 12:23:54 UTC+0000	
0x000000001d9d7b30	calc.exe	3216	2336	0x0000000017c87000	2018-10-11 12:23:58 UTC+0000	
0x000000001d9ebb30	notepad.exe	4080	2336	0x000000000fc38000	2018-10-11 12:23:57 UTC+0000	
0x000000001da11b30	explorer.exe	2336	2268	0x0000000015fec000	2018-10-11 12:23:40 UTC+0000	
0x000000001da1bb30	vmtoolsd.exe	2644	2336	0x000000001773d000	2018-10-11 12:23:41 UTC+0000	
0x000000001da45b30	CrossEXService	2460	1492	0x000000000cf6d000	2018-10-11 12:23:40 UTC+0000	
0x000000001dab9b30	taskeng.exe	2692	920	0x0000000009ae2000	2018-10-11 12:23:41 UTC+0000	
0x000000001dabb600	IMG5F50start_x	2752	1524	0x000000000081a000	2018-10-11 12:23:42 UTC+0000	
0x000000001dabbb30	IMG5F50start_x	2740	1524	0x0000000000ea1c000	2018-10-11 12:23:42 UTC+0000	
0x000000001daf8b30	AnySign4PC.exe	2840	2668	0x00000000005acb000	2018-10-11 12:23:42 UTC+0000	

탐지를 우회하는 악성코드에 대한 대응



Ticket Manager

1. 개요

Motivation



Bypass

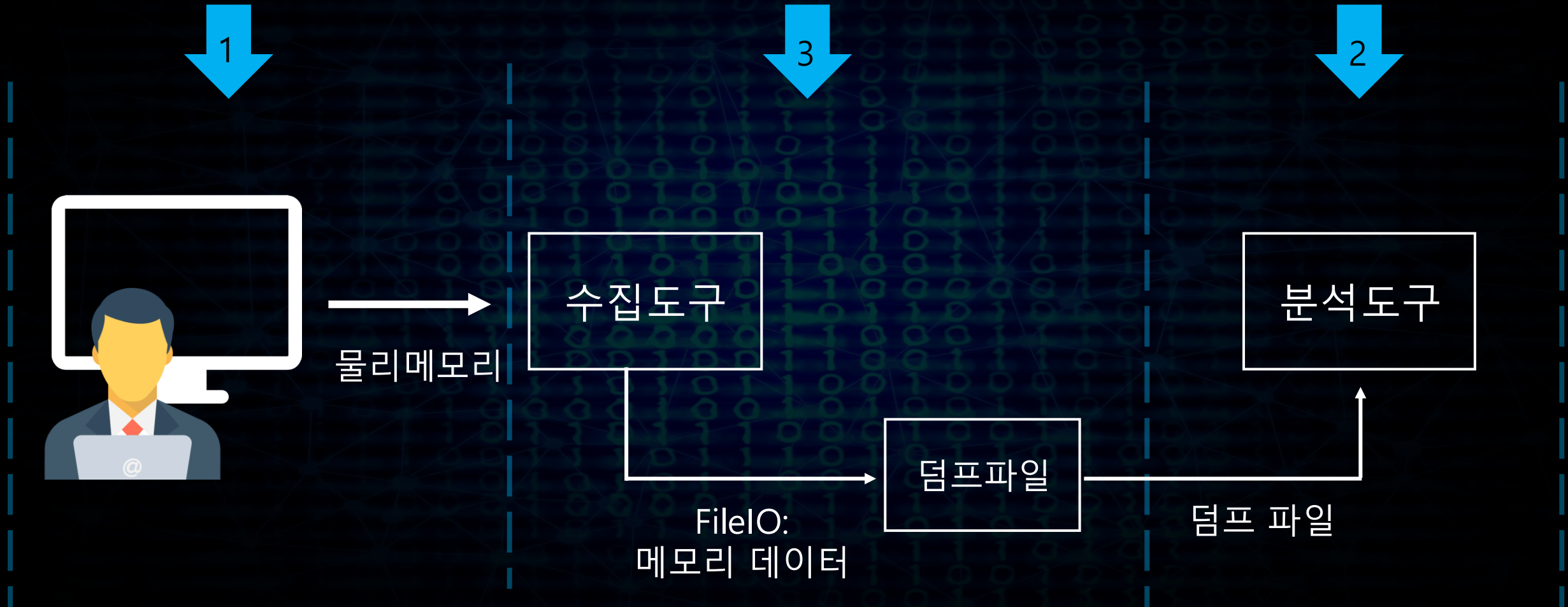


PoC
Implement

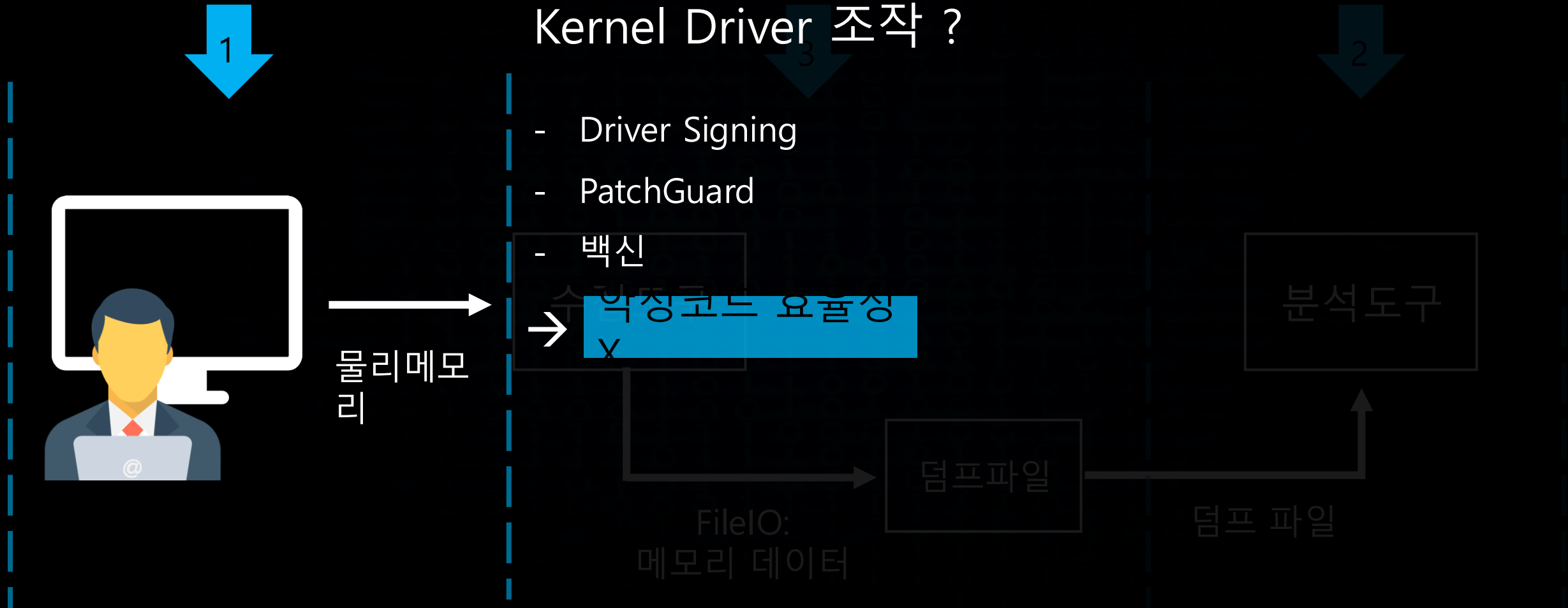


Tool
Detection

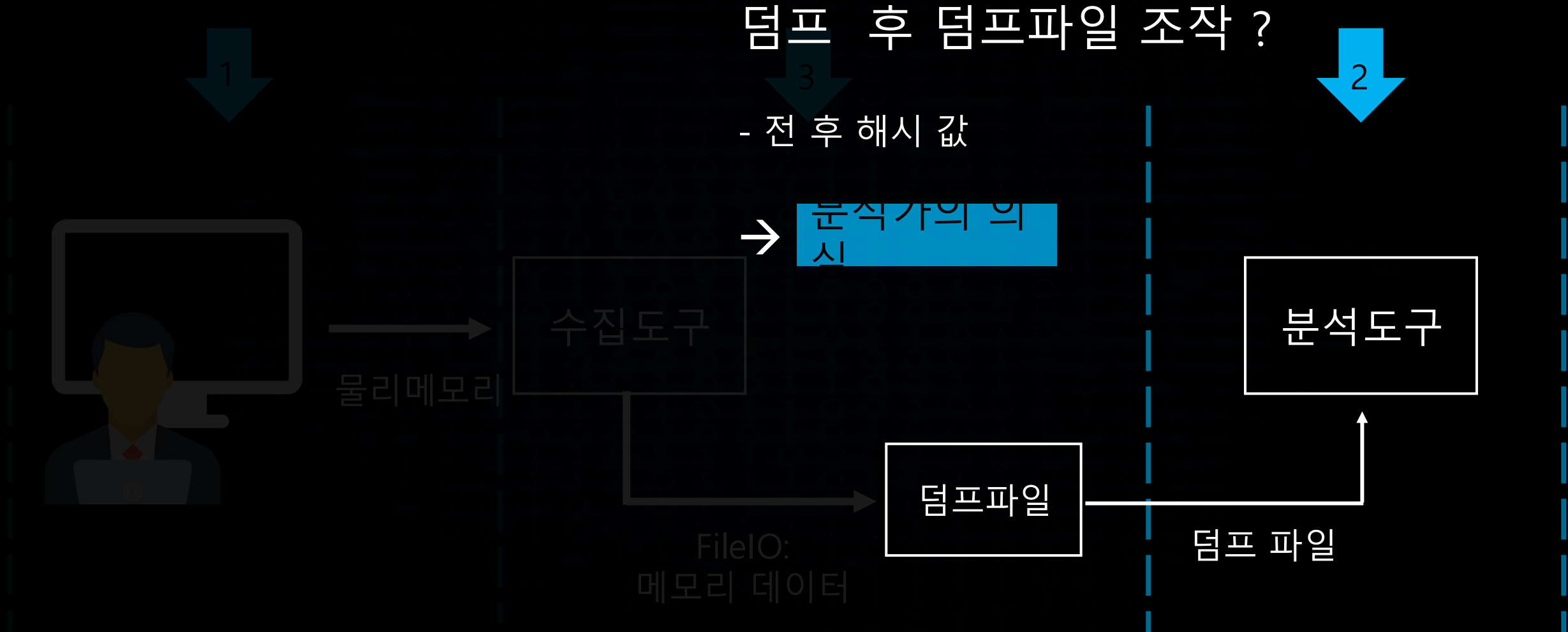
2. 방법론



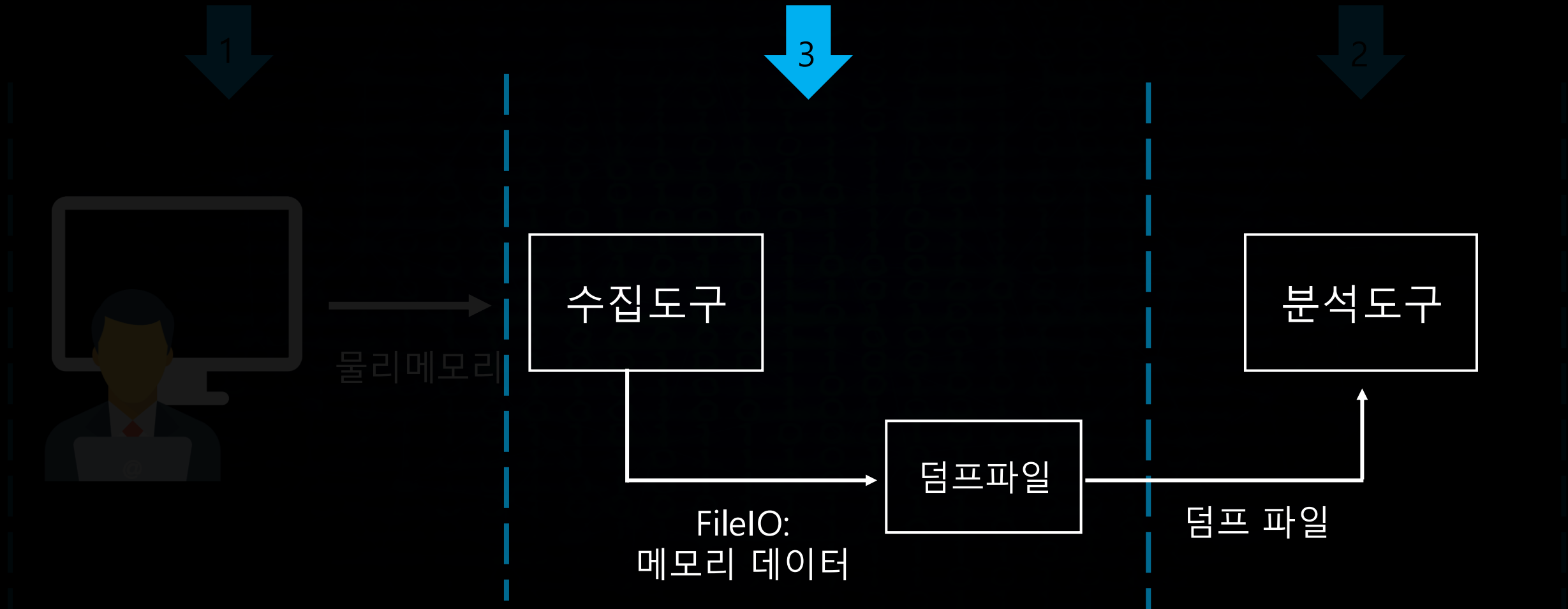
2. 방법론



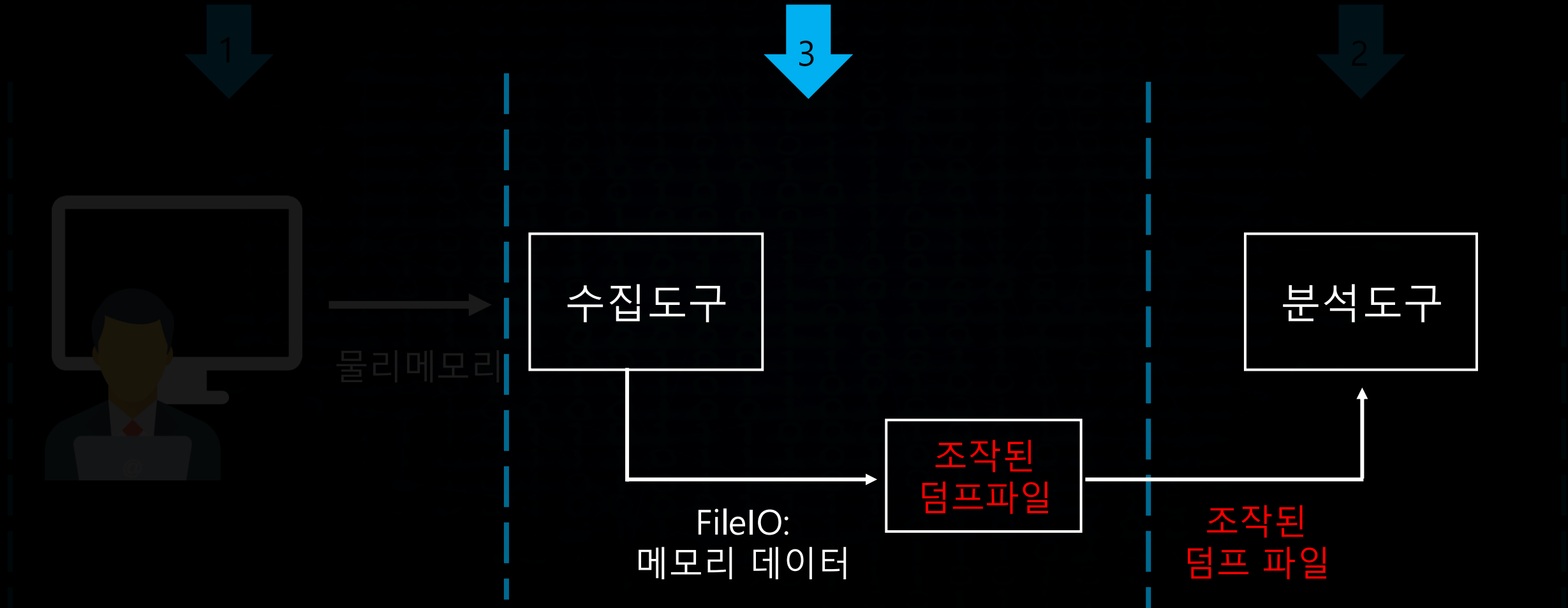
2. 방법론



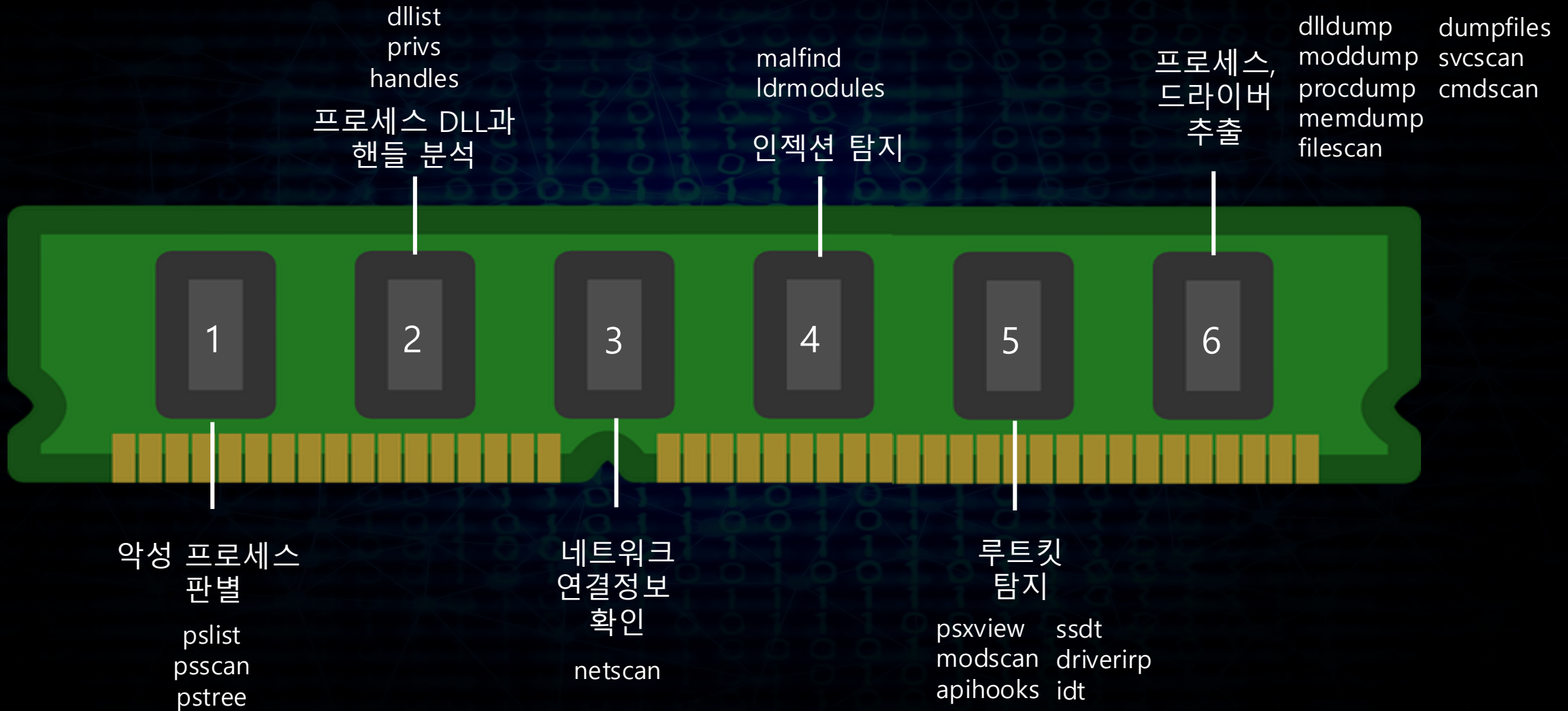
2. 방법론



2. 방법론



2. 방법론



2. 방법론

```
C:\#volatility-master>vol.py -f C:\test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name      PID  pslist psscan thrdproc pspcid csrss session deskthrd ExitTime
-----
0x000000001de37580 VGAAuthService. 1688 True  True  True  True  True  True  True
0x000000001e5f7b30 lsass.exe      512 True  True  True  True  True  True  False
0x000000001da1bb30 vmttoolsd.exe  2644 True  True  True  True  True  True  True
0x000000001e2c7060 svchost.exe    764 True  True  True  True  True  True  True
0x000000001da11b30 explorer.exe   2336 True  True  True  True  True  True  True
0x000000001e584b30 userinit.exe   2268 True  True  True  True  True  True  True
0x000000001e302710 svchost.exe    860 True  True  True  True  True  True  True
0x000000001de25b30 svchost.exe    1656 True  True  True  True  True  True  True
0x000000001e32b9e0 svchost.exe    896 True  True  True  True  True  True  True
0x000000001d603b30 malware.exe    3180 True  True  True  True  True  True  True
0x000000001d831280 wordpad.exe    3116 True  True  True  True  True  True  True
0x000000001df63a10 dwm.exe        2292 True  True  True  True  True  True  True
```

pslist

psscan

thrdproc

pspcid

csrss

session

deskthrd

7개의 필드 우회 -> 해당 프로세스 은닉

2. 방법론

```
>>> dt("_KDDEBUGGER_DATA64")
'_KDDEBUGGER_DATA64' (832 bytes)
0x0 : Header ['_DBGKD_DEBUG_DATA_HEADER64']
0x18 : KernBase ['unsigned long long']
0x20 : BreakpointWithStatus ['unsigned long long']
0x28 : SavedContext ['unsigned long long']
0x30 : ThCallbackStack ['unsigned short']
0x32 : NextCallback ['unsigned short']
0x34 : FramePointer ['unsigned short']
0x38 : KiCallUserMode ['unsigned long long']
0x40 : KeUserCallbackDispatcher ['unsigned long long']
0x48 : PslLoadedModuleList ['pointer', ['_LIST_ENTRY']]
0x50 : PsActiveProcessHead ['pointer', ['_LIST_ENTRY']]
0x58 : PspCidTable ['pointer', ['pointer', ['_PSP_CID_TABLE']]]
0x60 : ExpSystemResourcesList ['unsigned long long']
0x68 : ExpPagedPoolDescriptor ['unsigned long long']
```

pslist

psscan

thrdproc

pspcid

csrss

session

deskthrd

2. 방법론

```
class PoolScanProcess(poolscan.PoolScanner):
    """Pool scanner for process objects"""

    def __init__(self, address_space, **kwargs):
        poolscan.PoolScanner.__init__(self, address_space, **kwargs)

        self.struct_name = "_EPROCESS"
        self.object_type = "Process"
        # this allows us to find terminated processes
        self.skip_type_check = True
        self.pooltag = obj.VolMagic(address_space).ProcessPoolTag.v()
        size = 0x1ae # self.address_space.profile.get_obj_size("_EPROCESS")

        self.checks = [
            ('CheckPoolSize', dict(condition = lambda x: x >= size)),
            ('CheckPoolType', dict(paged = False, non_paged = True, free = True)),
            ('CheckPoolIndex', dict(value = lambda x: x < 5)),
        ]

class PSScan(common.AbstractScanCommand):
    """Pool scanner for process objects"""

    scanners = [PoolScanProcess]
```

pslist

psscan

thrdproc

pspcid

csrss

session

deskthrd

2. 방법론

```
116 def check_thrdproc(self, _addr_space):
117     """Enumerate processes indirectly by ETHREAD scanning"""
118     ret = dict()
119
120     for ethread in modscan.ThrdScan(self._config).calculate():
121         if ethread.ExitTime != 0:
122             continue
123         # Bounce back to the threads owner
124         process = None
125         if hasattr(ethread.Tcb, 'Process'):
126             process = ethread.Tcb.Process.dereference_as('_EPROCESS')
127         elif hasattr(ethread, 'ThreadsProcess'):
128             process = ethread.ThreadsProcess.dereference()
129         # Make sure the bounce succeeded
130         if (process and process.ExitTime == 0 and
131             process.UniqueProcessId > 0 and
132             process.UniqueProcessId < 65535):
133             ret[PsXview.get_file_offset(process)] = process
134
135     return ret
```

0x170	ExitTime
0x1e0	SessionProcessLinks
0x258	Win32Process



2. 방법론

```
def check_pspcid(self, addr_space):  
    """Enumerate processes by walking the PspCidTable"""  
    ret = dict()  
  
    # Follow the pointers to the table base  
    kdbg = tasks.get_kdbg(addr_space)  
    PspCidTable = kdbg.PspCidTable.dereference().dereference()  
  
    # Walk the handle table  
    for handle in PspCidTable.handles():  
        if handle.get_object_type() == "Process":  
            process = handle.dereference_as("_EPROCESS")  
            ret[PsXview.get_file_offset(process)] = process  
  
    return ret
```

```
def check_csrss_handles(self, all_tasks):  
    """Enumerate processes using the csrss.exe handle table"""  
    ret = dict()  
  
    for p in all_tasks:  
        if str(p.ImageFileName).lower() == "csrss.exe":  
            # Gather the handles to process objects  
            for handle in p.ObjectTable.handles():  
                if handle.get_object_type() == "Process":  
                    process = handle.dereference_as("_EPROCESS")  
                    ret[PsXview.get_file_offset(process)] = process  
  
    return ret
```

pslist

psscan

thrdproc

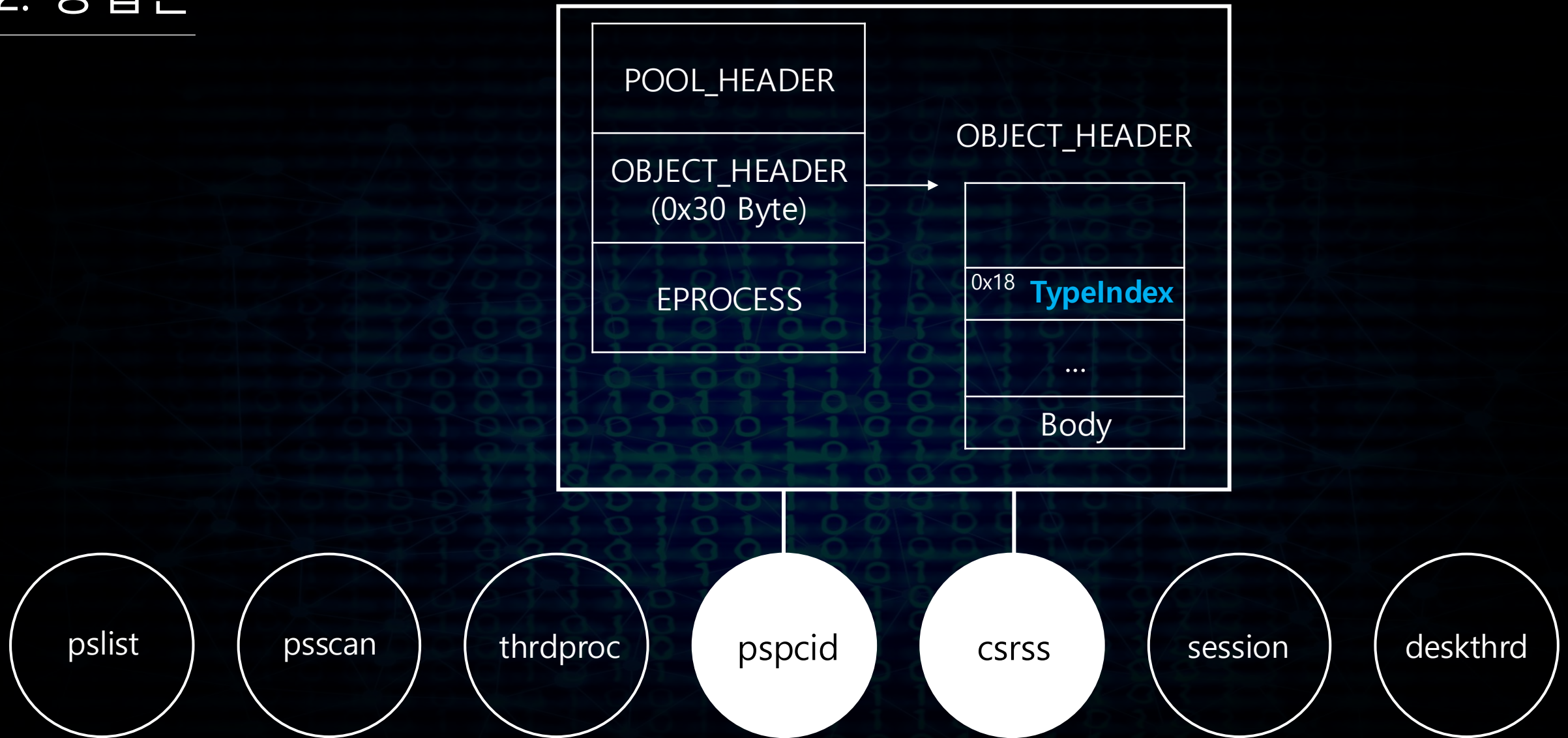
pspcid

csrss

session

deskthrd

2. 방법론



2. 방법론

```
def check_desktop_thread(self, addr_space):  
    """Enumerate processes from desktop threads"""  
  
    ret = dict()  
    for windowstation in windowstations.WndScan(self._config).calculate():  
        for desktop in windowstation.desktops():  
            for thread in desktop.threads():  
                process = thread.ppi.Process.dereference()  
                if process == None:  
                    continue  
                ret[PsXview.get_file_offset(process)] = process
```

pslist

psscan

thrdproc

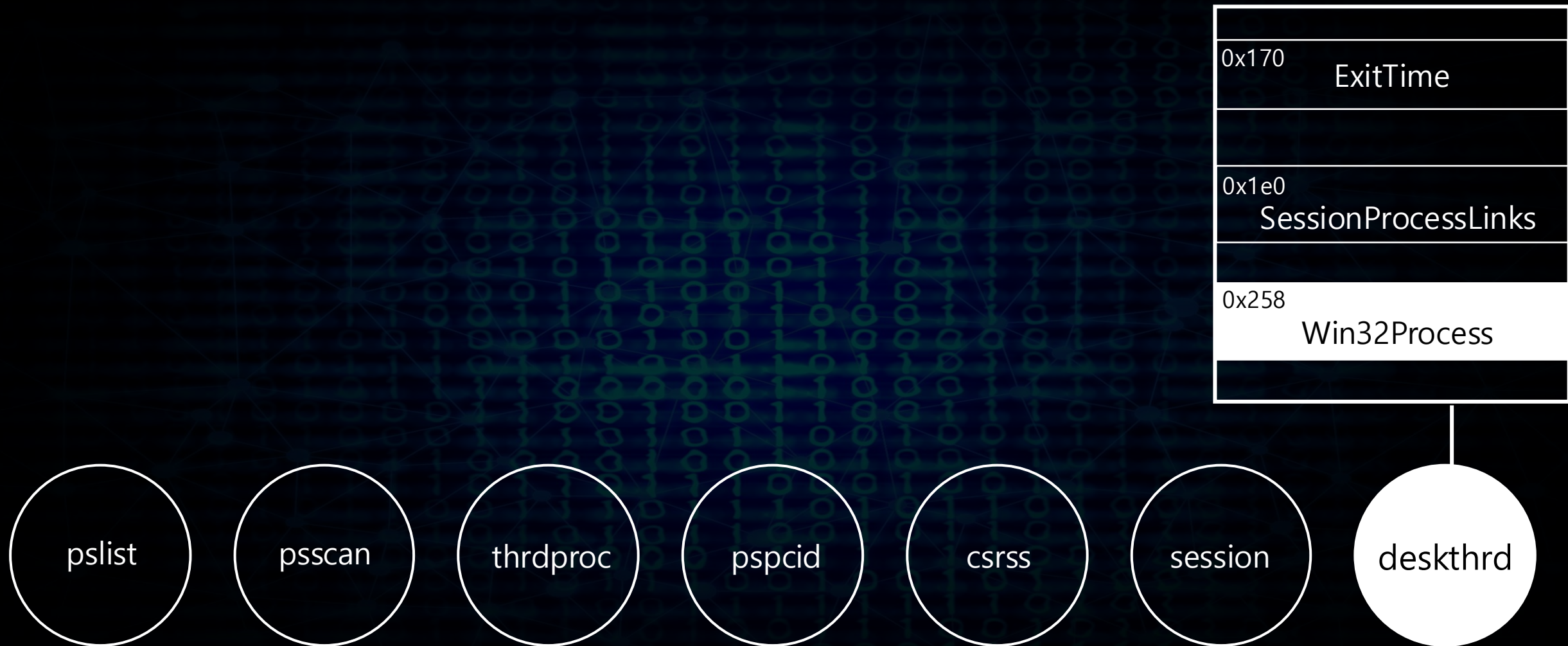
pspcid

csrss

session

deskthrd

2. 방법론

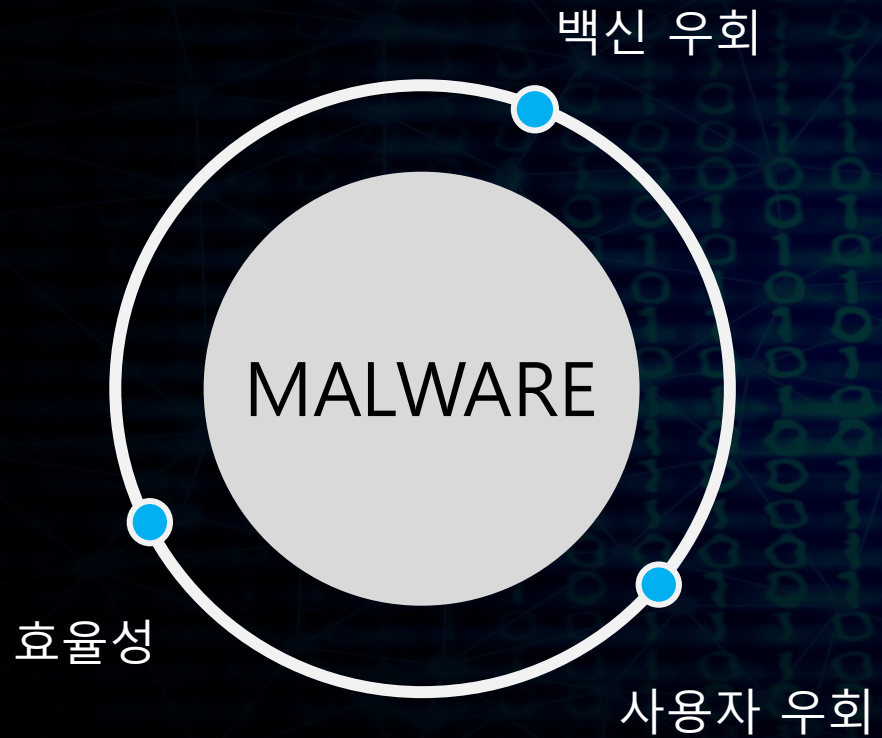


2. 방법론

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name      PID  pslist  psscan  thrdproc  pspcid  csrss  session  deskthrd  ExitTime
-----
0x000000001de37580 VGAAuthService. 1688 True    True    True    True    True    True    True    True
0x000000001e5f7b30 lsass.exe      512  True    True    True    True    True    True    True    False
0x000000001da1bb30 vmtoolsd.exe   2644 True    True    True    True    True    True    True    True
0x000000001e2c7060 svchost.exe    764  True    True    True    True    True    True    True    True
0x000000001da11b30 explorer.exe   2336 True    True    True    True    True    True    True    True
0x000000001e584b30 userinit.exe   2268 True    True    True    True    True    True    True    True
0x000000001e302710 svchost.exe    860  True    True    True    True    True    True    True    True
0x000000001de25b30 svchost.exe    1656 True    True    True    True    True    True    True    True
0x000000001e32b9e0 svchost.exe    896  True    True    True    True    True    True    True    True
0x000000001d603b30 malware.exe    3180 True    True    True    True    True    True    True    True
0x000000001d831280 wordpad.exe    3116 True    True    True    True    True    True    True    True
0x000000001df63a10 dwm.exe        2292 True    True    True    True    True    True    True    True
```

```
C:\#volatility-master>vol.py -f C:\#test.vmem --profile=Win7SP1x64 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)      Name      PID  pslist  psscan  thrdproc  pspcid  csrss  session  deskthrd  ExitTime
-----
0x000000001de37580 VGAAuthService. 1688 True    True    True    True    True    True    True    True
0x000000001e5f7b30 lsass.exe      512  True    True    True    True    True    True    True    False
0x000000001da1bb30 vmtoolsd.exe   2644 True    True    True    True    True    True    True    True
0x000000001e2c7060 svchost.exe    764  True    True    True    True    True    True    True    True
0x000000001da11b30 explorer.exe   2336 True    True    True    True    True    True    True    True
0x000000001e584b30 userinit.exe   2268 True    True    True    True    True    True    True    True
0x000000001e302710 svchost.exe    860  True    True    True    True    True    True    True    True
0x000000001de25b30 svchost.exe    1656 True    True    True    True    True    True    True    True
0x000000001e32b9e0 svchost.exe    896  True    True    True    True    True    True    True    True
0x000000001d831280 wordpad.exe    3116 True    True    True    True    True    True    True    True
0x000000001df63a10 dwm.exe        2292 True    True    True    True    True    True    True    True
0x000000001e078220 spoolsv.exe    1204 True    True    True    True    True    True    True    True
```

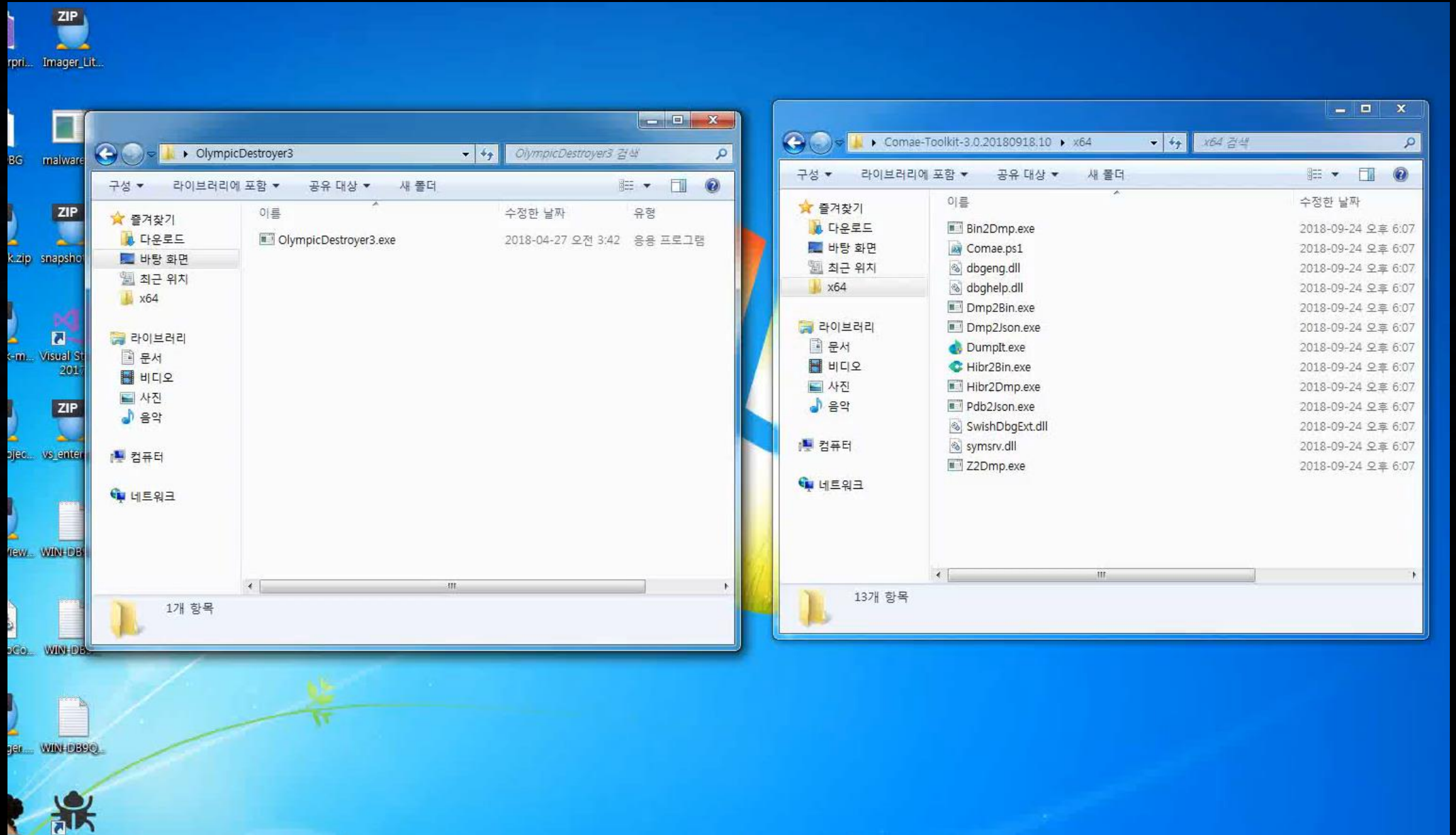
2. 방법론



Hooking 을 통한
특정 메모리 영역 조작
(Live)

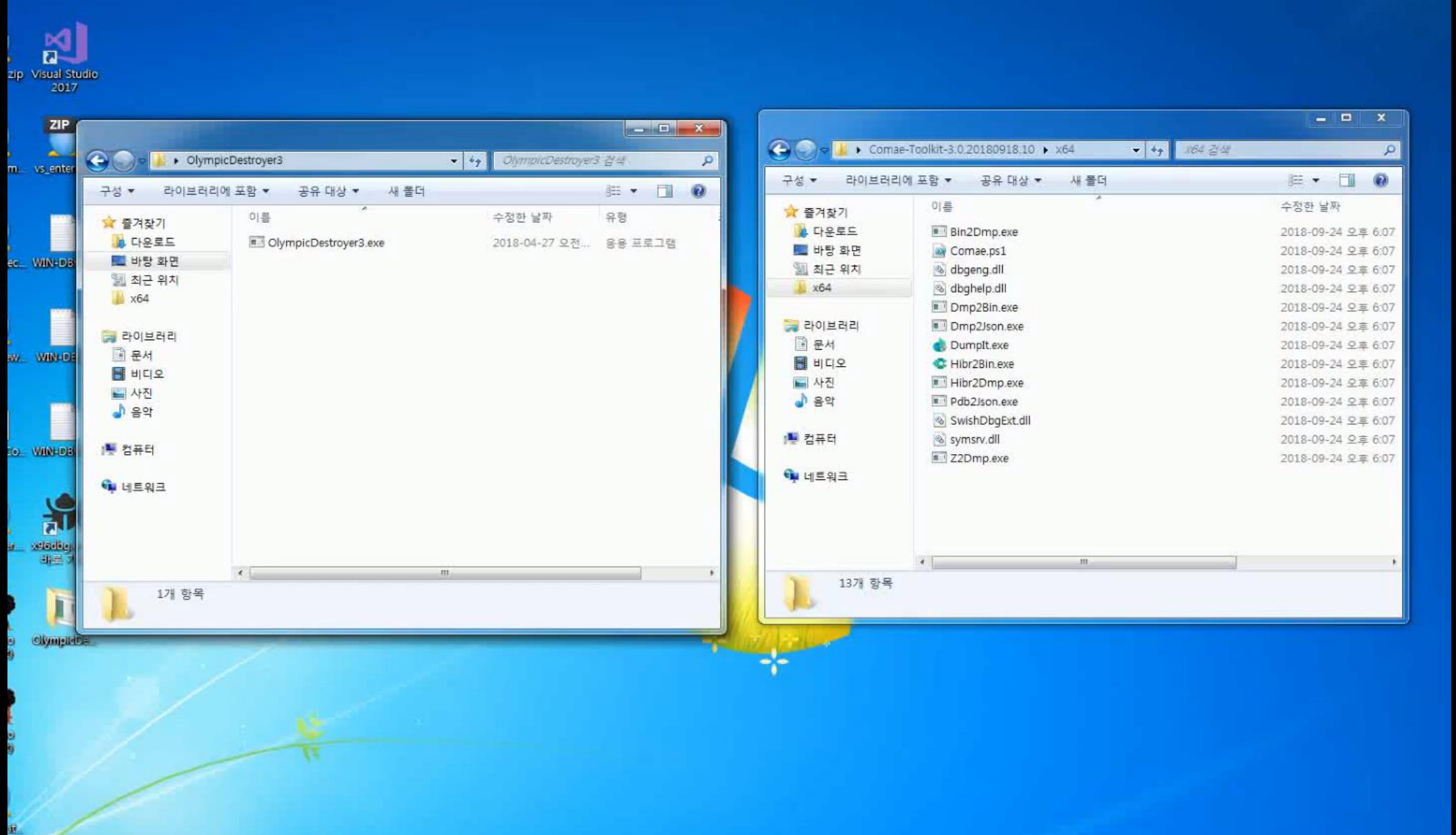
3. PoC

DEMO
(정상)



3. PoC

DEMO
(우회)



4. 방안 제시

procdump



" OlympicDestroyer.exe "



1876

OlympicDestroyer.exe

notepad.exe

4. 방안 제시

procdump

```
def generator(self, data):
    for task in data:
        task_space = task.get_process_address_space()
        if task_space == None:
            result = "Error: Cannot acquire process AS"
        elif task.Peb == None:
            # we must use m() here, because any other attempt to
            # reference task.Peb will try to instantiate the _PEB
            result = "Error: PEB at {0:#x} is unavailable (possibly due to paging)".format(task.m('Peb'))
        elif task_space.vtop(task.Peb.ImageBaseAddress) == None:
            result = "Error: ImageBaseAddress at {0:#x} is unavailable (possibly due to paging)".format(task.Peb.ImageBaseAddress)
        else:
            dump_file = "executable." + str(task.UniqueProcessId) + ".exe"
            result = self.dump_pe(task_space,
                                  task.Peb.ImageBaseAddress,
                                  dump_file)
        yield (0,
              [Address(task.obj_offset),
               Address(task.Peb.ImageBaseAddress),
               str(task.ImageFileName),
               str(result)])
```

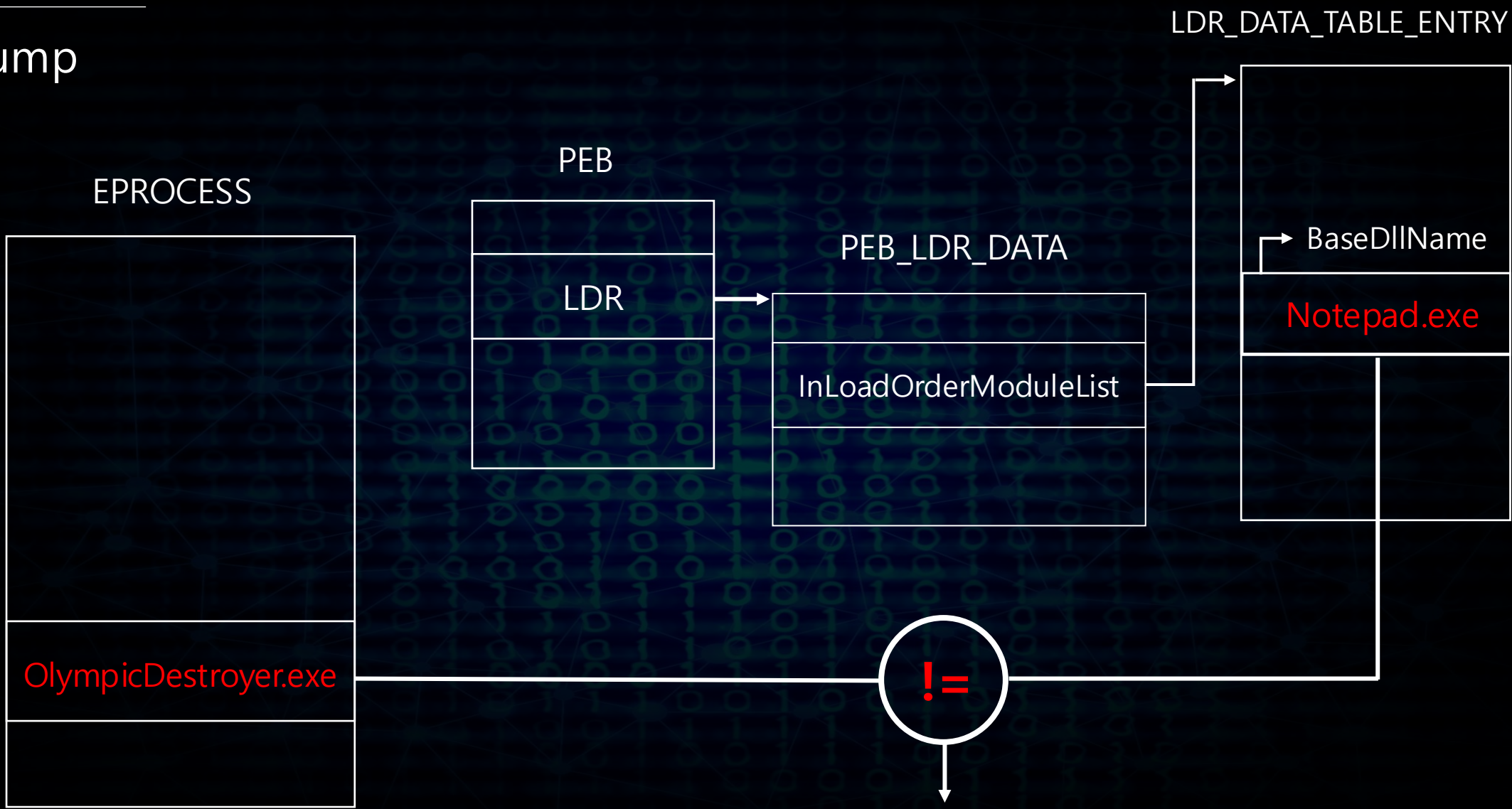
(1) Address Space

(2) PEB

(3) ImageBaseAddress

4. 방안 제시

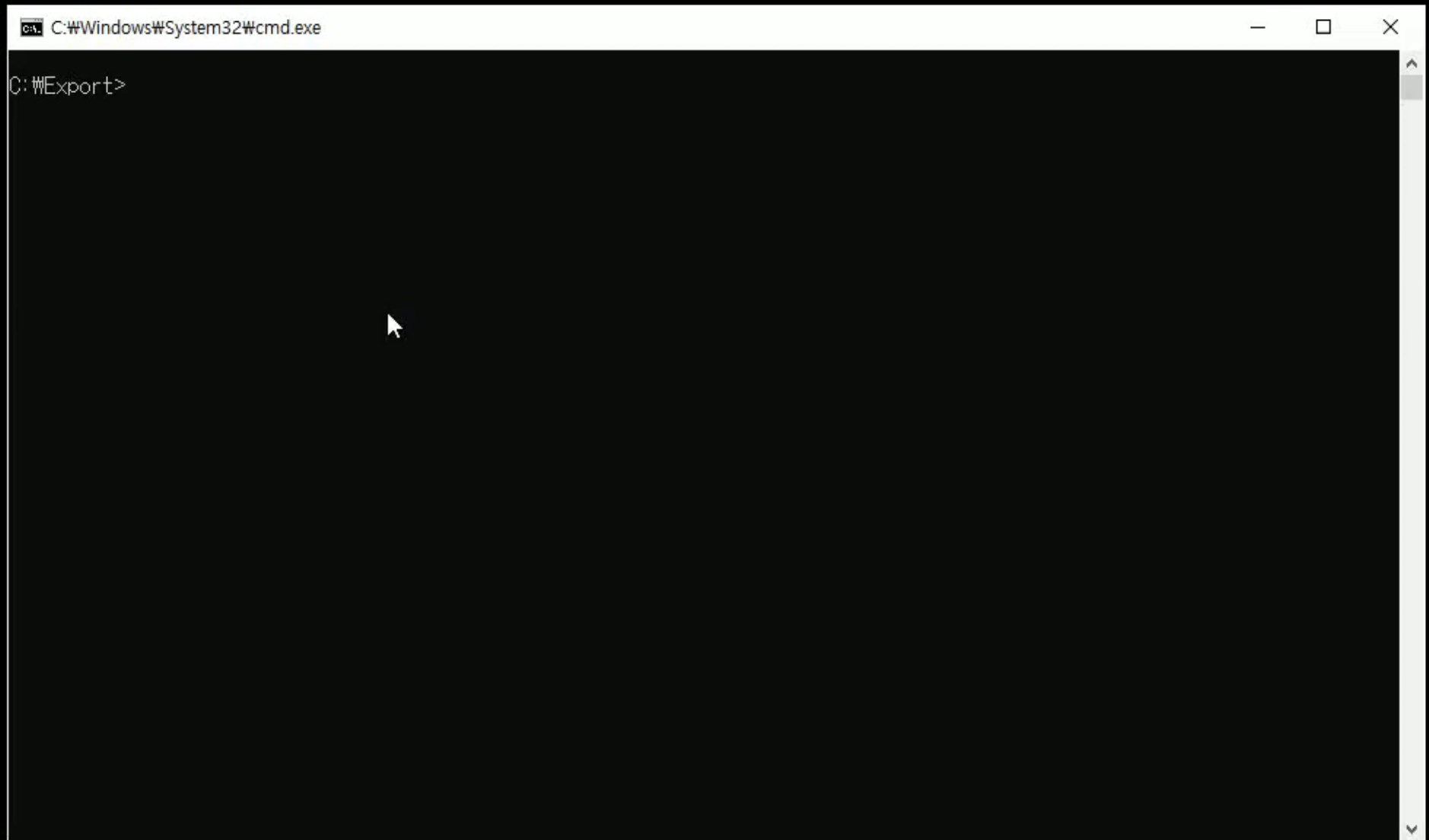
procdump



조작된 프로세스

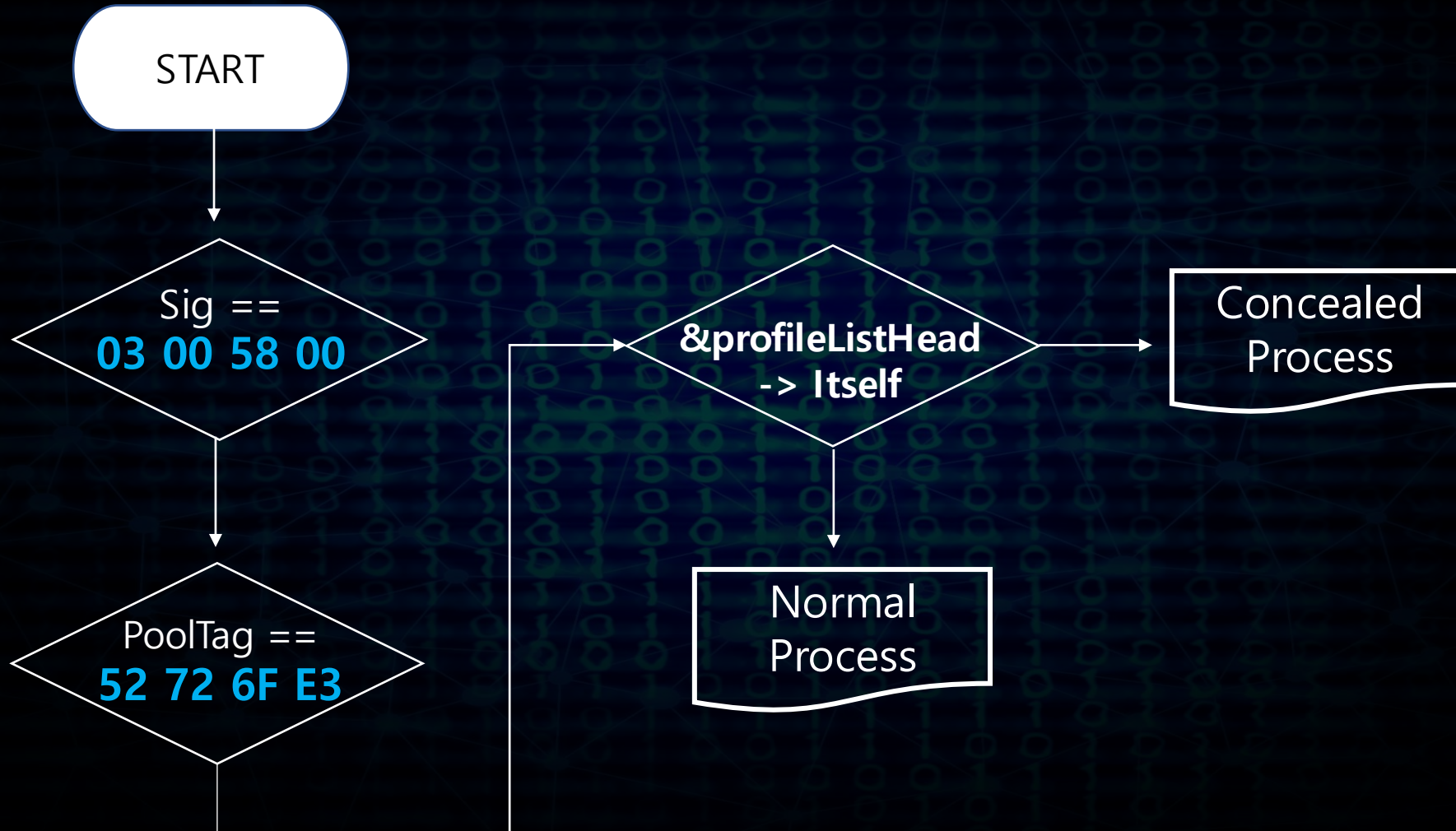
4. 방안 제시

procdump



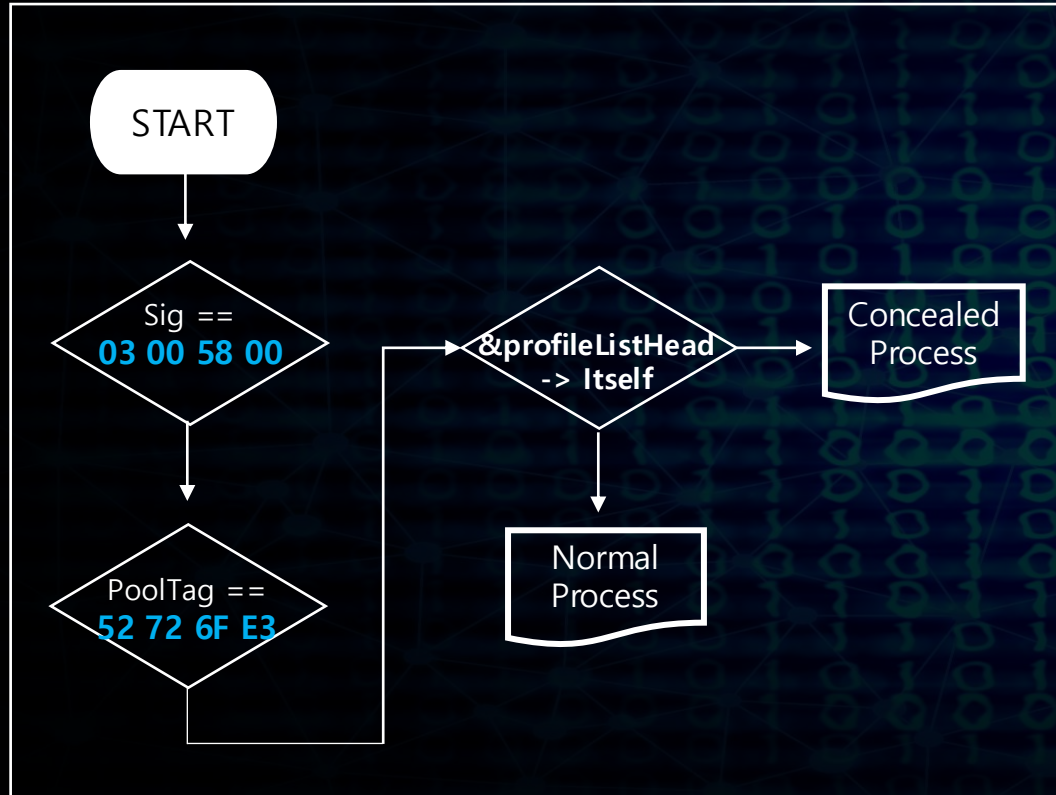
4. 방안 제시

psscan



4. 방안 제시

psscan



```
class PoolTagCheck(scan.ScannerCheck):  
    """ This scanner checks for the occurrence of a pool tag """  
    def __init__(self, address_space, tag = None, **kwargs):  
        scan.ScannerCheck.__init__(self, address_space, **kwargs)  
        self.tag = tag  
  
    def skip(self, data, offset):  
        try:  
            nextval = data.index(self.tag, offset + 1)  
            return nextval - offset  
        except ValueError:  
            ## Substring is not found - skip to the end of this data buffer  
            return len(data) - offset  
  
    def check(self, offset):  
        data = self.address_space.read(offset, len(self.tag))  
        return data == self.tag
```

4. 방안 제시

psscan

```
[+] 프로세스 탐색을 시작합니다
[-] 파일 크기에 따라 탐색 시간이 길어질 수 있습니다...

OFFSET(P) : 0x1d603b30 DTB : 0xbbcf000 PID : 3180 PPID : 656 NAME : malware.exe
OFFSET(P) : 0x1d610b30 DTB : 0x10a9c000 PID : 3556 PPID : 2544 NAME : ipconfig.exe
OFFSET(P) : 0x1d831280 DTB : 0x15e06000 PID : 3116 PPID : 656 NAME : wordpad.exe
OFFSET(P) : 0x1d837b30 DTB : 0x123c2000 PID : 3168 PPID : 2856 NAME : node.exe
OFFSET(P) : 0x1e2aeb30 DTB : 0xa38c000 PID : 720 PPID : 496 NAME : vmacthlp.exe
OFFSET(P) : 0x1e2c7060 DTB : 0xa098000 PID : 764 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e302710 DTB : 0x9ad4000 PID : 860 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e32b9e0 DTB : 0x9760000 PID : 896 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e333b30 DTB : 0x9027000 PID : 920 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e35e7b0 DTB : 0x8642000 PID : 1016 PPID : 860 NAME : audiodg.exe
OFFSET(P) : 0x1e38e9b0 DTB : 0x86f1000 PID : 396 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e3dfb30 DTB : 0x72bc000 PID : 916 PPID : 496 NAME : svchost.exe
OFFSET(P) : 0x1e55c060 DTB : 0xdf8d000 PID : 436 PPID : 340 NAME : wininit.exe
OFFSET(P) : 0x1e561060 DTB : 0xdd12000 PID : 444 PPID : 428 NAME : csrss.exe
OFFSET(P) : 0x1e5733b0 DTB : 0x143f2000 PID : 2152 PPID : 496 NAME : dllhost.exe
OFFSET(P) : 0x1e584b30 DTB : 0x1bb59000 PID : 2268 PPID : 540 NAME : userinit.exe
OFFSET(P) : 0x1e5cab30 DTB : 0xc394000 PID : 496 PPID : 436 NAME : services.exe
OFFSET(P) : 0x1e5f7b30 DTB : 0xc1f4000 PID : 512 PPID : 436 NAME : lsass.exe
OFFSET(P) : 0x1e5f9160 DTB : 0xc53a000 PID : 520 PPID : 436 NAME : lsm.exe
OFFSET(P) : 0x1e6e9b30 DTB : 0xe807000 PID : 356 PPID : 340 NAME : csrss.exe
OFFSET(P) : 0x1ef10b30 DTB : 0x13843000 PID : 268 PPID : 4 NAME : smss.exe
OFFSET(P) : 0x1ef5d060 DTB : 0x7f8a000 PID : 2508 PPID : 1564 NAME : MalWebCFAgent_
OFFSET(P) : 0x1fc0db30 DTB : 0xd0d1000 PID : 2772 PPID : 2692 NAME : GoogleUpdate.e
OFFSET(P) : 0x1ffbdb30 DTB : 0x187000 PID : 4 PPID : 0 NAME : System
```

```
-----탐색 결과 보고-----
[+] 정상 프로세스 60개, 은폐 프로세스 3개 임니다.
[-] 은폐된 프로세스는 ['malware.exe'] 임니다.
[-] 은폐된 프로세스는 ['ipconfig.exe'] 임니다.
[-] 은폐된 프로세스는 ['userinit.exe'] 임니다.
```

ARRESTED

그 외

filescan

[+] 파일 목록 탐색을 시작합니다
[-] 파일 크기에 따라 탐색 시간이 길어질 수 있습니다...

Offset(P)	Offset(F)	PATH
0x1d629c80	0x86eff80	\\Windows\\System32\\schedcli.dll
0x1d62bf20	0x15702240	\\Windows\\System32\\rpcss.dll
0x1d62de10	0x144ba150	\\Windows\\System32\\winusb.dll
0x1d630f20	0xc05b510	\\Windows\\System32\\d3d9.dll
0x1d631290	0x9fa9820	\\Windows\\System32\\rasapi32.dll
0x1d63abf0	0x18fb8930	\\Windows\\System32\\msacm32.dll
0x1d650320	0x1ba93ec0	\\Windows\\System32\\PortableDeviceTypes.dll
0x1d653310	0x8537f50	\\Windows\\SysWOW64\\ow64.dll
0x1d672db0	0x3ce320	\\Windows\\System32\\Syncreg.dll
0x1d67af20	0x19cd5c30	\\Windows\\System32\\d3d11.dll
0x1d684210	0x1376c990	\\Windows\\System32\\drmvm2clt.dll
0x1d6b2710	0x5338e30	\\Windows\\System32\\AltTab.dll
0x1d6b3670	0x1ad45340	\\Windows\\System32\\DXP.dll
0x1d6c11e0	0x19cbc530	\\Windows\\System32\\stobject.dll
0x1d6c1dd0	0xdf1e8f0	\\Windows\\System32\\batmeter.dll
0x1d6c3c80	0xbf122e0	\\Windows\\SysWOW64\\dhcpcsvc.dll
0x1d6cd240	0x2468b20	\\Windows\\System32\\netshell.dll
0x1d6df8e0	0x144c1740	\\Windows\\System32\\mssprxy.dll
0x1fb06f20	0xe402490	\\Windows\\System32\\aspnet_counters.dll
0x1fb13ba0	0x15efe40	\\Windows\\SysWOW64\\urlmon.dll
0x1feb7450	0x20a2900	\\Windows\\System32\\credui.dll
0x1ff30d10	0x1453e300	\\Windows\\System32\\MsCtfMonitor.dll

```
[+] 으메 파일 오브젝트는 3개 입니다.
[-] Offset(P) : 0x1dd629c80 | PATH : \\Windows#System32#schedcli.dll
[-] Offset(P) : 0x1dd6cf20 | PATH : \\Windows#System32#gameux.dll
[-] Offset(P) : 0x1e252590 | PATH : \\Windows#System32#dnsapi.dll
```


4. 방안 제시

교차 검증 도구



Volatility와 Rekall 의 교차 검증 결과

를

Web으로 구현

4. 방안 제시

교차 검증 도구



pslist : 프로세스 목록 출력

Cross check result <Different : 2 Identical : 57>

process	pid	volatility	rekall
System	4	O	O
smss.exe	268	O	O
csrss.exe	356	O	O
svchost.exe	396	O	O
wininit.exe	436	O	O
csrss.exe	444	O	O
services.exe	496	O	O
lsmd.exe	520	O	O
svchost.exe	656	O	O
vmacthlp.exe	720	O	O
svchost.exe	764	O	O
svchost.exe	860	O	O
svchost.exe	896	O	O
svchost.exe	920	O	O
audiodg.exe	1016	O	O
spoolsv.exe	1204	O	O
AnySign4PCLaun	1332	X	O
ObCrossEXServ	1492	O	O
IMG5F50Svc.exe	1524	O	O

Cross check result <Different : 2 Identical : 57>

process	pid	volatility	rekall
System	4	O	O

Pid 860

C:\\Windows\\System32\\svchost.exe C:\\Windows\\SYSTEM32\\ntdll.dll C:\\Windows\\system32\\kernel32.dll
C:\\Windows\\system32\\KERNELBASE.dll C:\\Windows\\system32\\msvcrt.dll C:\\Windows\\SYSTEM32\\sechost.dll
C:\\Windows\\system32\\RPCRT4.dll C:\\Windows\\system32\\ole32.dll C:\\Windows\\system32\\GDI32.dll
C:\\Windows\\system32\\USER32.dll C:\\Windows\\system32\\LPK.dll C:\\Windows\\system32\\USP10.dll
C:\\Windows\\system32\\IMM32.DLL C:\\Windows\\system32\\MSCTF.dll C:\\Windows\\System32\\CRYPTBASE.dll
C:\\Windows\\system32\\ADVAPI32.dll c:\\windows\\system32\\wevtsvc.dll C:\\Windows\\System32\\RpcRtRemote.dll
C:\\Windows\\System32\\secur32.dll C:\\Windows\\System32\\SSPICLI.DLL C:\\Windows\\System32\\credssp.dll
C:\\Windows\\system32\\WS2_32.dll C:\\Windows\\system32\\NSI.dll C:\\Windows\\system32\\mswsock.dll
C:\\Windows\\System32\\wshtcpip.dll C:\\Windows\\System32\\wship6.dll C:\\Windows\\System32\\GPAPI.dll
c:\\windows\\system32\\audiosrv.dll c:\\windows\\system32\\POWRPROF.dll C:\\Windows\\system32\\SETUPAPI.dll
C:\\Windows\\system32\\CFGMR32.dll C:\\Windows\\system32\\OLEAUT32.dll C:\\Windows\\system32\\DEVOBJ.dll
c:\\windows\\system32\\MMDevAPI.DLL c:\\windows\\system32\\PROPSYS.dll c:\\windows\\system32\\AVRT.dll
C:\\Windows\\system32\\CLBCatQ.DLL C:\\Windows\\System32\\WINSTA.dll C:\\Windows\\system32\\SHLWAPI.dll
C:\\Windows\\System32\\CRYPTSP.dll C:\\Windows\\system32\\rsaenh.dll C:\\Windows\\System32\\audioses.dll
c:\\windows\\system32\\lmhsvc.dll c:\\windows\\system32\\PHLPAPI.DLL c:\\windows\\system32\\WINNSI.DLL
c:\\windows\\system32\\nrpsrv.DLL c:\\windows\\system32\\dhcpcore.dll c:\\windows\\system32\\DNSAPI.dll
C:\\Windows\\System32\\firewallapi.dll C:\\Windows\\System32\\VERSION.dll C:\\Windows\\System32\\dhcpcore6.dll
C:\\Windows\\System32\\dhcpcsvc6.DLL C:\\Windows\\System32\\dhcpcsvc.DLL c:\\windows\\system32\\provsvc.dll
C:\\Windows\\System32\\npmproxy.dll C:\\Windows\\system32\\actxprxy.dll C:\\Windows\\system32\\FunDisc.dll
C:\\Windows\\system32\\ATL.DLL C:\\Windows\\System32\\msxml6.dll C:\\Windows\\system32\\fdproxy.dll
C:\\ProgramFiles\\InternetExplorer\\ieproxy.dll

svchost.exe	920	O	O
audiodg.exe	1016	O	O
csrss.exe	1204	O	O

4. 방안 제시

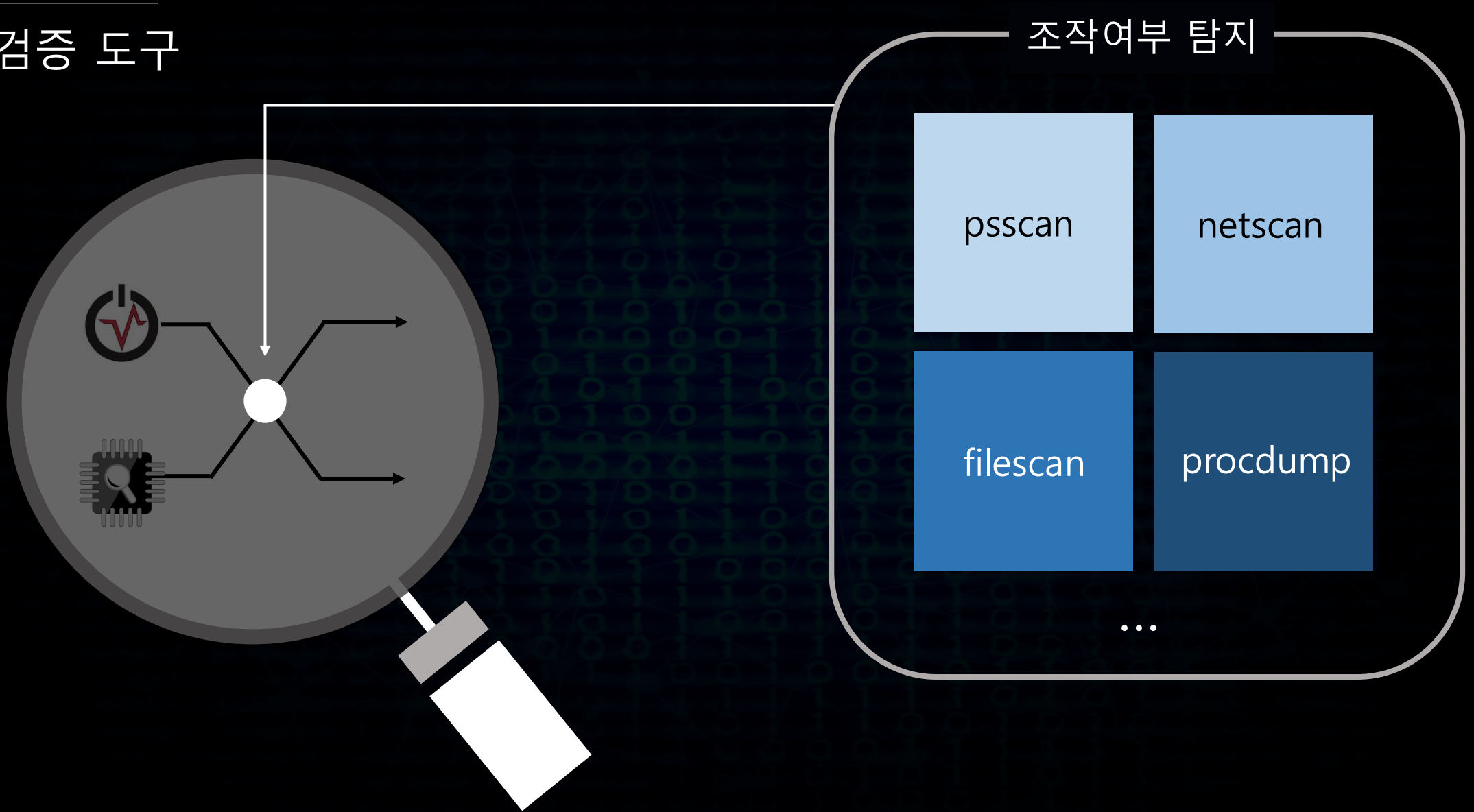
교차 검증 도구

MalGround - 교차 검증 도구입니다.



4. 방안 제시

교차 검증 도구



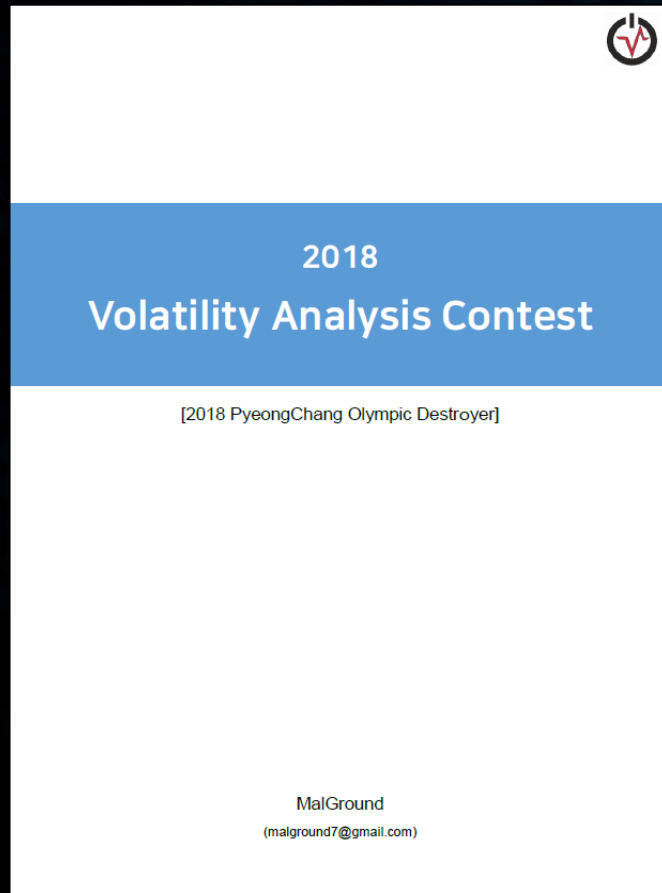
학술대회 발표



5. 결과

VAC

2018 Volatility Analysis Contest



Analysis Contest

1st: 2018 VAC Report by Team Decepticon (South Korea)

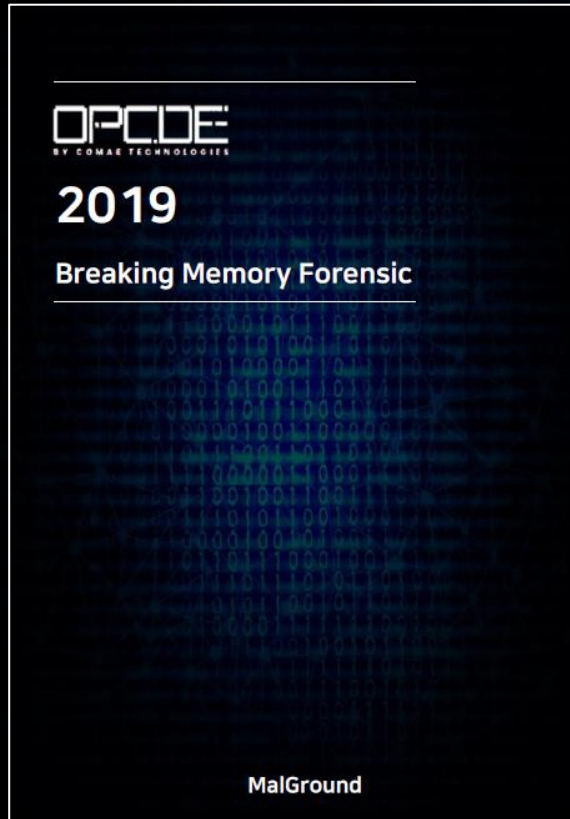
The authors of this report put together a realistic lab scenario modeled after Korean APT investigations they have performed. We were not only impressed by the number of Volatility plugins represented in the analysis efforts, but also that the infected systems spanned multiple operating systems (Windows and Linux). Memory analysis was leveraged to shed light on the toolkits and methodologies used by the attackers, including Eternal Blue, Dark Comet, Spear Phishing, HWP exploits, DLL injections, MongoDB vulnerabilities, and more. Evidence from Outlook PSTs were reconstructed from RAM and shellcode was explored and identified in memory using Yarascan, Volshell, and various other capabilities provided by Volatility.

2nd: 2018 PyeongChang Olympic Destroyer by Team **MalGround** (South Korea)

This analysis report, written by the MalGround team from South Korea, describes a scenario based on the Olympic Destroyer events surrounding the 2018 PyeongChang Winter Olympic Games. Before the Olympics officially began, the attackers attempted to disrupt the opening ceremonies by intercepting the event's critical computer systems and infrastructure offline. The cyber attack was initiated with a targeted spear phish and then leveraged a "network worm" to propagate using network shares and stolen passwords. The simulated scenario involved three systems: an Active Directory server running Windows Server 2008 R2 Standard 64-bit, a victim PC running Windows 7 Ultimate K x86, and an attacker PC running Kali Linux 64-bit. In the simulated scenario, a file-less malware attack, leveraging Empire (Mimikatz, BypassUAC), is combined with the actual Olympic Destroyer malware. The authors leveraged memory analysis to find supporting temporal artifacts, identify suspicious characteristics of processes, and extract memory resident strings and executables. This submission includes the analysts' report and a memory sample from the Windows 7 machine where the Olympic Destroyer malware was executed.

5. 결과

CFP



5. 결과

CFP

For those concerned DFRWS

Ground Mal <malground7@gmail.com>
eu-papers에게 ▾

Hi this is team MalGround, We are working on topics related to memory analysis bypass method.

Specifically, when collecting memory, The [redacted] modifies certain parts of the file being dumped.

As a result, we succeeded in bypassing [redacted] process, thread, and network connection in [redacted] a PoC.

Previous previous studies,
[Takahiro Haruyama, One-byte [redacted] BlackHat EUROPE, 2012]
[Luka Milkovic, Defeating Windows [redacted] 2012]
[Jake Williams and Alissa Torres, [redacted] through Memory Disarray, shmoocon, 2014]
we have supplemented the limit of [redacted]

We would like to submit a simple demo on [redacted] interested in this, please reply :)

Thanks
Team MalGround

← 답장

➡ 전달

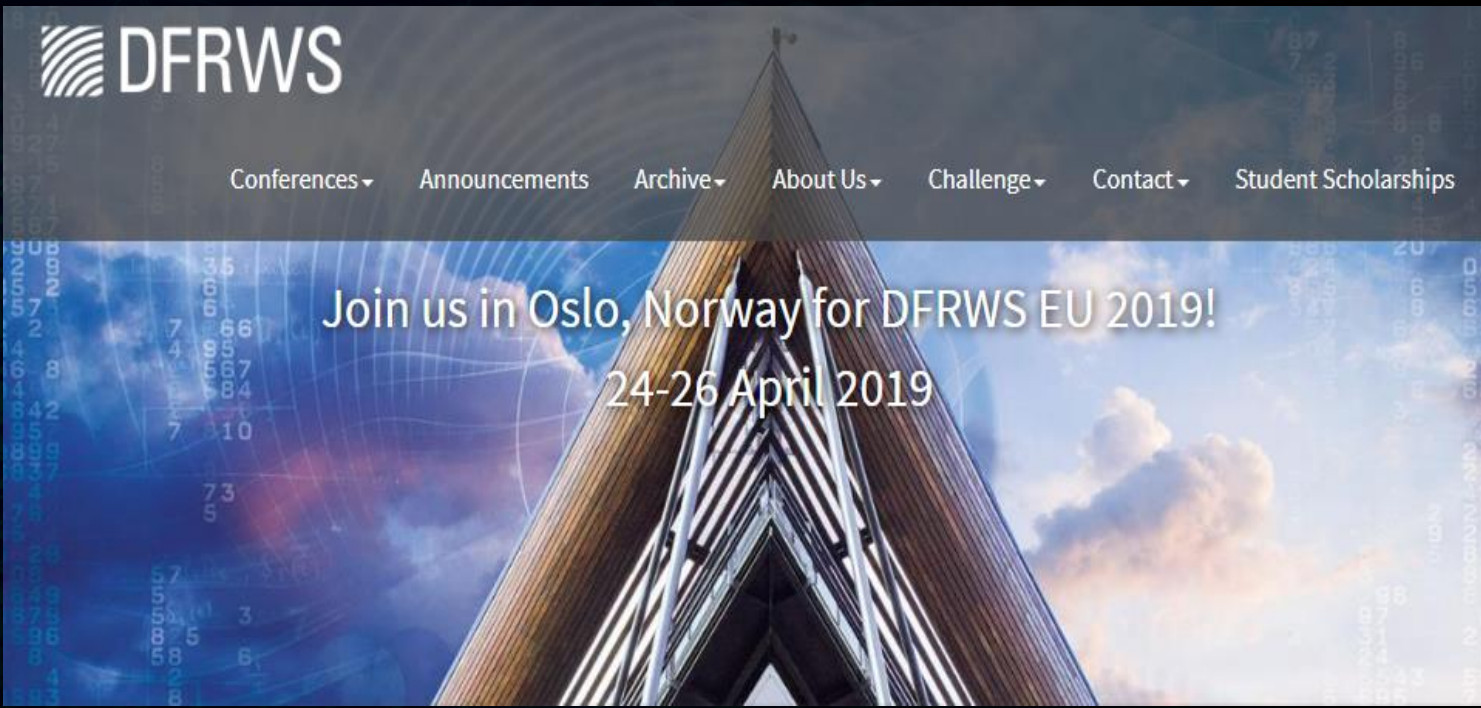


 DFRWS

Conferences ▾ Announcements Archive ▾ About Us ▾ Challenge ▾ Contact ▾ Student Scholarships

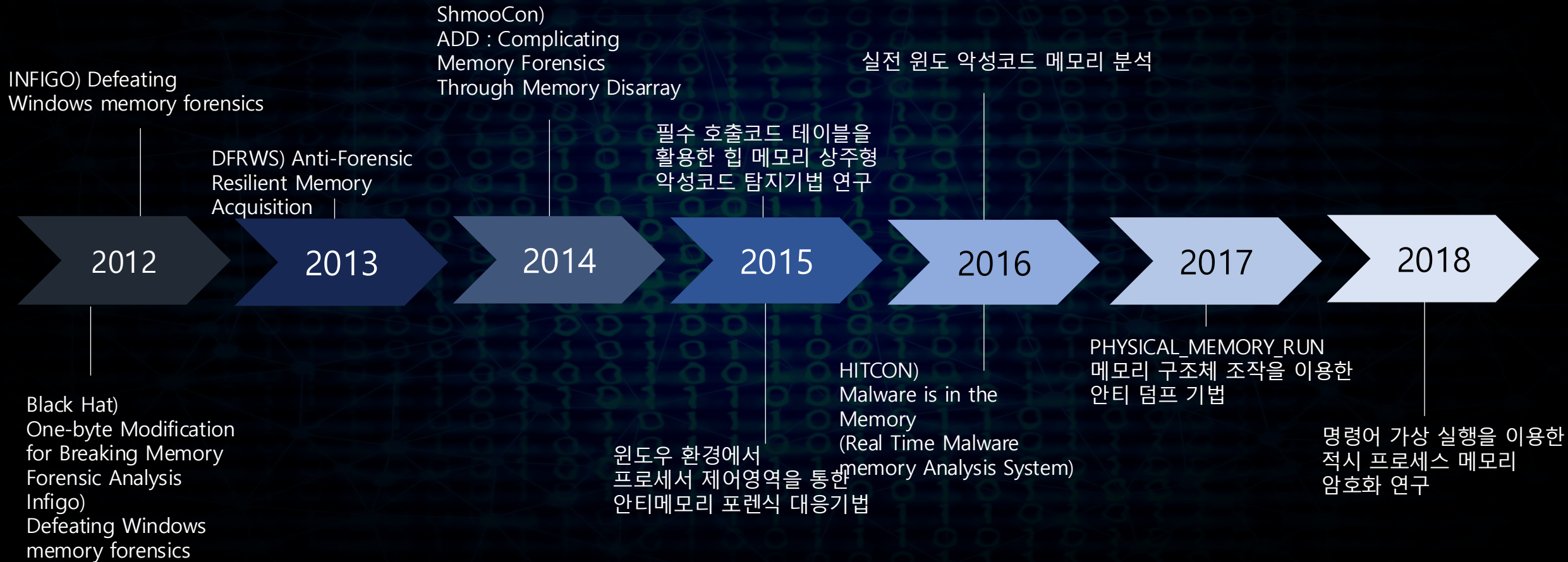
Join us in Oslo, Norway for DFRWS EU 2019!

24-26 April 2019



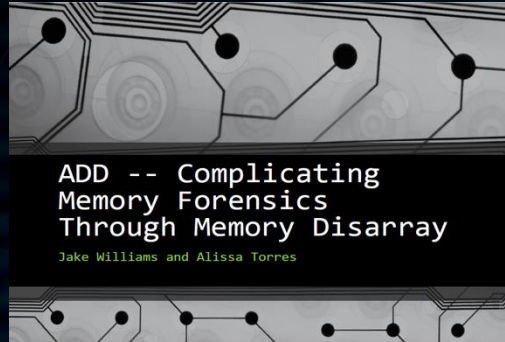
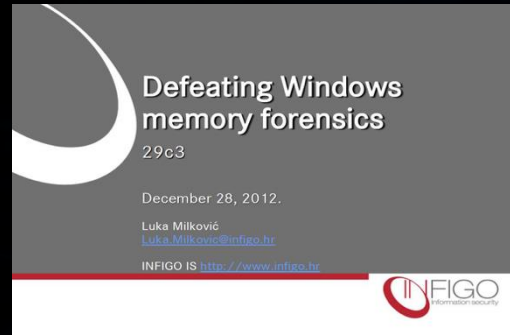
5. 결과

선행 연구



5. 결과

선행 연구



2012

2013

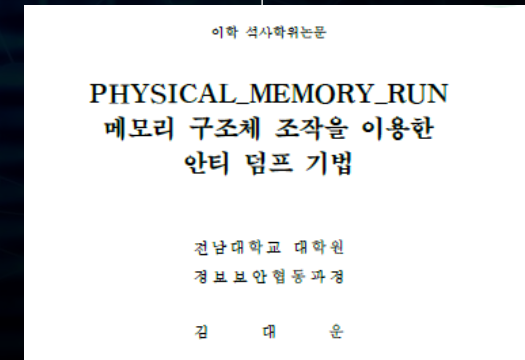
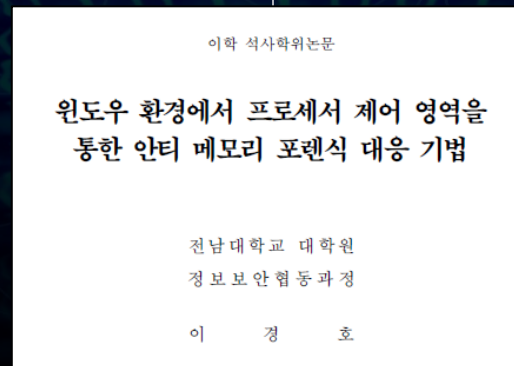
2014

2015

2016

2017

2018



5. 결과

외부 협력



심영진 멘토님



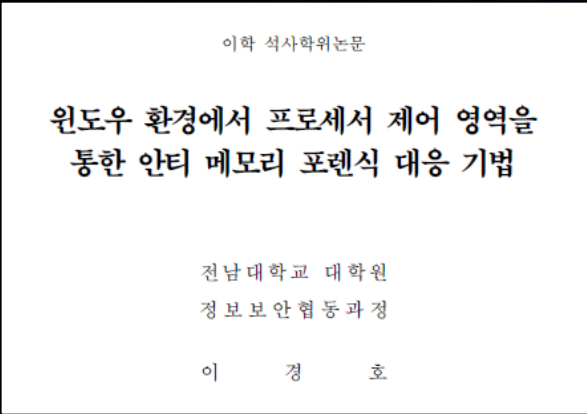
이경식 연구원님



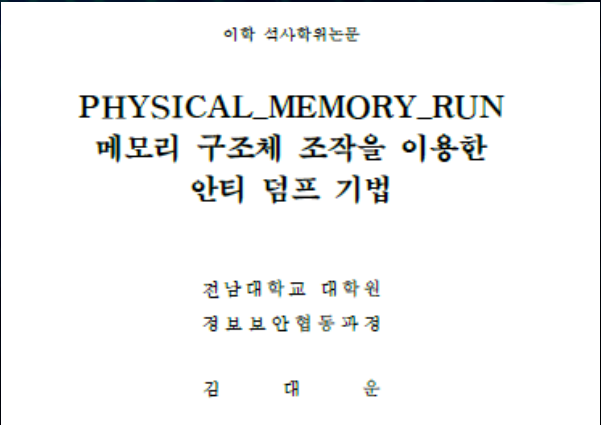
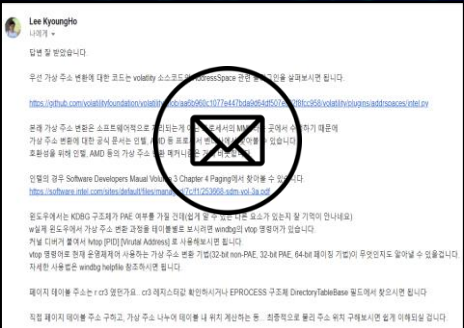
노용한 멘토님



이준형 연구원님



이경호 연구원님

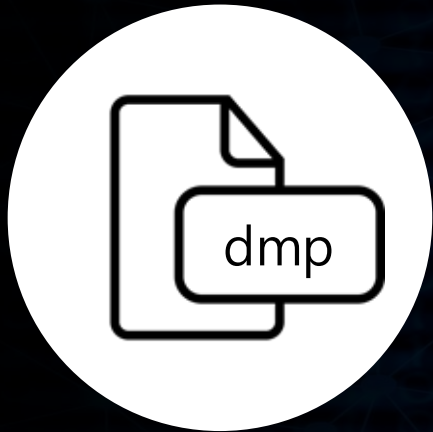


김대운 연구원님



5. 결과

추가 연구 사항



Dumplt 외의
다른 도구
추가 진행



Process
memory
dump



교차검증
도구
고도화



Contribution



2019
Volatility
Plugin Contest

6. 역할

분석우회



이한얼



전은진

- PM (프로젝트 관리)
- 분석도구 우회 방안 연구 (Volatility, Rekall...)
- 주소 변환 코드 제작
- 탐지방안 코드 제작 (netscan, psscan, filesan)
- 분석도구 우회방안 연구 (Volatility, Rekall ...)
- 주소 변환 과정 탐색
- 탐지방안 코드 제작 (procdump)
- 문서 관리
- 발표자료 제작

개발



김태우



한지혁



김한솔

- PoC 코드 개발 총괄
- 수집도구 분석 및 후킹 (DumpIt)
- PoC 코드 테스트
- 주소 변환 과정 탐색
- 산출물 제작
- 공격기법 연구
- 수집도구 분석 및 후킹 (DumpIt)
- PoC 코드 개발
- 교차 검증 코드 개발
- 공격기법 보고서 작성
- 공격기법 연구
- 교차검증 도구 Web 구현
- 수집도구 분석 및 후킹 (FTK Imager)

7. WBS

일 정	착 수		수행		결과
	9월	10월	11월	12월	
대분류	주제 구체화 및 사전조사	우회 기법 연구		논문 및 PoC 제작	대응 방안
상세 Task	- 주제 관련 논문 조사 - Volatility Contest 준비	수집방해 분석방해		논문 및 PoC 제작 교차검증	- 제작한 PoC 탐지 방안 연구 - 결과보고서작성 - CFP - 교차검증 도구
		<수집방해> - 후킹 테스트 코드 개발 <분석방해> - 메모리 분석 도구 심층분석 - 우회 리스트제작		논문 및 PoC 제작 - 논문 작성 - 교차검증 - 백업본 기반 PoC 설계 - PoC 구현 - 도구 우회 여부 테스트	
산출물	- 프로젝트계획서 - 수행계획서 - 참고논문보고서	- Volatility Contest 보고서 - 분석도구 우회 방안 보고서 - 수집도구 우회 방안 보고서 - 기타 우회 방안 보고서		- Volatility Command 보고서 - 논문 - PoC 테스트 계획 보고서 - PoC 테스트 결과 보고서	- 개선방안보고서 - 최종결과보고서



감사합니다.

